TypeScript — Complete Beginner Guide

Detailed classroom lecture for students preparing for modern JavaScript frameworks

From JavaScript fundamentals to the TypeScript features used heavily in NestJS

Learning Objectives

Understand TypeScript and its relationship with JavaScript.

Write variables, arrays, objects, functions and classes with types.

Use interfaces, type aliases, unions, optional properties and generics.

Understand constructors, access modifiers and inheritance.

Use async/await with Promise<T>.

Build the foundation required for NestJS.

Approach

Concept → Syntax → Example → Common mistake → Practice

Students should type the examples rather than only reading them.

Recommended prerequisite: basic JavaScript.

1. What is TypeScript?

Definition, purpose and relationship with JavaScript

TypeScript is a strongly typed programming language built on top of JavaScript. It adds a static type system and developer tooling, then is compiled/transpiled to JavaScript.

```
JavaScript
    ↓
TypeScript adds types + tooling
    ↓
TypeScript compiler
    ↓
JavaScript
    ↓
Browser / Node.js
```

Important points

- TypeScript is not a replacement runtime for JavaScript.
- Browsers normally execute JavaScript, not raw TypeScript.
- TypeScript catches many type-related mistakes before execution.
- It is widely used for large applications and backend development.

2. Why Do We Need TypeScript?

The problem with dynamically typed JavaScript

```
let age = 25;
age = "twenty five";

function add(a, b) {
    return a + b;
}

add(10, "20");
```

JavaScript may allow values of different types to flow through the program. Some mistakes are discovered only when the code runs.

TypeScript's benefit

let age: number = 25;

age = "twenty five"; // ✕

The editor/compiler can warn us before the program is executed.

3. TypeScript vs JavaScript

Understand the main difference

JavaScript

```
let value = 10;
value = "Hello"; // allowed
```

Dynamic typing

Type is associated with the current value at runtime.

TypeScript

```
let value: number = 10;
value = "Hello"; // ✕
```

Static type checking

Types can be checked during development/compilation.

Key idea: TypeScript helps developers find many mistakes earlier.

4. Basic Types

The most important primitive types

```
let name: string = "Hamim";
let age: number = 25;
let cgpa: number = 3.98;
let isTeacher: boolean = true;
```

string

Text values.

Example: "Hamim", "CSE", "Hello"

number

Integer and floating-point numbers.

Example: 25, 3.98, -10

boolean

Logical true/false values.

Example: true, false

Other types you will encounter: null, undefined, object, arrays, functions, union types, custom types.

5. Type Annotation

How to explicitly tell TypeScript a variable's type

Syntax:

```
let variableName: type = value;
```

```
let studentName: string = "Hamim";
let studentAge: number = 25;
let passed: boolean = true;

studentAge = "25"; // ✕
```

Read the syntax as:

studentAge : number

“studentAge must contain a number.”

The colon (:) is used to specify a type.

Classroom tip: Explain the colon as “is of type”.

6. Type Inference

TypeScript can determine types automatically

```
let name = "Hamim";    // inferred: string
let age = 25;          // inferred: number
let active = true;     // inferred: boolean
```

What happened?

TypeScript looks at the assigned value and infers the most appropriate type.

```
let age = 25;
age = "25"; // ✕
```

Inference reduces unnecessary repetition, but explicit types are valuable when defining reusable structures and APIs.

7. Arrays

Typing collections of values

```
let names: string[] = [
    "Rahim",
    "Karim",
    "Hamim"
];

names.push("Sakib"); // ✓
names.push(100);       // ✕
```

Common syntax

```
let marks: number[] = [80, 75, 90];

let names2: Array<string> = [
    "A", "B", "C"
];
```

string[] means: every element should be a string.

number[] means: every element should be a number.

8. Objects

Representing related data

```
const student = {
    id: 42017,
    name: "Hamim",
    cgpa: 3.98
};

console.log(student.name);
```

An object stores related properties together.

id → 42017

name → "Hamim"

cgpa → 3.98

For small objects, TypeScript can infer the structure. For reusable contracts, define an interface or type.

9. Interface — The Blueprint

A central TypeScript concept for object structure

An interface defines the shape an object should follow. It does not create the actual object.

```
interface Student {
    id: number;
    name: string;
    cgpa: number;
}

const student: Student = {
    id: 42017,
    name: "Hamim",
    cgpa: 3.98
};
```

Think of it as a contract

A Student must have:

id → number

name → string

cgpa → number

The actual object provides the data.

10. Interface — What if the object is wrong?

Type checking in action

```
interface Student {
    id: number;
    name: string;
    cgpa: number;
}

const student: Student = {
    id: 42017,
    name: "Hamim",
    cgpa: "3.98" // ✕
};
```

Two common errors

1. Wrong type

cgpa should be number, but "3.98" is a string.

2. Missing property

If cgpa is not included, TypeScript reports an error because it is required.

Interfaces make data contracts visible and easier to maintain.

11. Optional Properties (?)

Some object properties are not mandatory

```
interface Student {
    id: number;
    name: string;
    cgpa?: number;
}

const s1: Student = {
    id: 1,
    name: "Hamim"
}; // ✓
```

The ? means “optional”.

cgpa?: number

The object may contain cgpa, but it does not have to.

```
const s2: Student = {
    id: 2,
    name: "Rahim",
    cgpa: 3.80
}; // ✓
```

12. Functions — Parameters & Return Types

Making functions type-safe

```
function add(a: number, b: number): number {
    return a + b;
}

add(10, 20);        // ✅
add("10", "20"); // ❌
```

Function anatomy

a: number → first parameter must be a number

b: number → second parameter must be a number

: number → function must return a number

```
function printName(name: string):
void {
    console.log(name);
}
```

13. Interface + Function

Passing structured data to functions

```
interface Student {
    id: number;
    name: string;
    cgpa: number;
}

function displayStudent(student: Student): void {
    console.log(student.name);
    console.log(student.cgpa);
}
```

What TypeScript knows

The function accepts a Student.

Therefore student.name, student.id and student.cgpa are known properties with known types.

This reduces accidental misuse.

This pattern becomes very important in backend services and controllers.

14. Union Types (|)

When a value can have more than one allowed type

```
let id: number | string;

id = 42017;       // ✓
id = "42017";     // ✓
id = true;        // ✗
```

The | symbol means OR.

number | string

means:

number OR string

```
function printId(id: number | string):
void {
    console.log(id);
}
```

15. Literal Union Types

Restrict a value to specific choices

```
type Status = "active" | "inactive";

let status: Status;

status = "active";    // ✓
status = "inactive";  // ✓
status = "deleted";   // ✗
```

Why is this useful?

Instead of allowing any string, we allow only known valid values.

Common examples:

- status
- role
- direction
- environment
- operation type

16. type Alias vs interface

Two ways to describe and name types

```
interface Student {
    id: number;
    name: string;
}
```

```
type Student = {
    id: number;
    name: string;
};
```

For basic object structures, both can look very similar.

```
type ID = number | string;
type Status = "active" | "inactive";
```

Rule of thumb: interface is commonly used for object/class contracts; type is especially flexible for unions, literals and type composition.

17. Classes

Blueprints for creating objects with data and behavior

```
class Student {
    name: string;
    age: number;

    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }

    introduce(): void {
        console.log(this.name);
    }
}
```

Create an object

```
const student = new Student(
    "Hamim",
    25
);

student.introduce();
```

A class can contain properties, constructors and methods.

18. Constructors

A constructor runs when an object is created

```
class Student {
    name: string;
    age: number;

    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }
}

const s = new Student("Hamim", 25);
```

Execution flow

new Student("Hamim", 25)

↓

constructor(name, age)

↓

this.name = "Hamim"

this.age = 25

TypeScript shortcut:

```
constructor(
  public name: string,
  public age: number
) {}
```

19. Access Modifiers

Control who can access class members

```
class Student {
    public name: string;
    private cgpa: number;
    protected id: number;

    constructor(...) {}
}
```

public

Accessible from anywhere that has access to the object.

private

Accessible only inside the class where it is declared.

protected

Accessible inside the class and classes that inherit from it.

Encapsulation idea: keep internal details protected and expose only what users of the class need.

20. Example: private Data + Public Method

Why access modifiers are useful

```
class BankAccount {
    private balance: number;

    constructor(balance: number) {
        this.balance = balance;
    }

    deposit(amount: number): void {
        this.balance += amount;
    }

    getBalance(): number {
        return this.balance;
    }
}
```

```
const account = new
BankAccount(1000);

account.deposit(500);

console.log(account.getBalance());
// 1500

// account.balance ✕
```

The balance is controlled through methods instead of being directly changed from outside.

21. Inheritance: extends

Reuse properties and methods from another class

```
class Person {
    constructor(public name: string) {}

    introduce(): void {
        console.log(`My name is ${this.name}`);
    }
}

class Student extends Person {
    constructor(
        name: string,
        public studentId: number
    ) {
        super(name);
    }
}
```

extends

Student inherits from Person.

Student can use introduce() even though the method is defined in Person.

super(name)

Calls the parent class constructor so the inherited part is initialized.

22. async / await

Asynchronous programming for APIs and databases

An async function returns a Promise. await lets us wait for the Promise result inside an async function.

```
async function getName(): Promise<string> {
    return "Hamim";
}

async function main(): Promise<void> {
    const name = await getName();
    console.log(name);
}
```

Promise<string>

The function will eventually produce a string.

await

Waits for the Promise result before continuing.

Very common in backend programming: database queries, HTTP requests, file operations and external APIs.

23. Promise<T>

Understanding the type of an asynchronous result

```
interface Student {
    id: number;
    name: string;
}

async function getStudent(): Promise<Student> {
    return {
        id: 42017,
        name: "Hamim"
    };
}
```

Read Promise<Student> as:

"This asynchronous operation will eventually give me a Student."

```
Promise<Student[]>

= eventually gives an array of Students
```

24. Generics <T>

Reusable code that remains type-safe

A generic is a placeholder for a type. The actual type is supplied when the function/type is used.

```
function getData<T>(data: T): T {
    return data;
}

const name = getData("Hamim"); // string
const age = getData(25);          // number
const ok = getData(true);         // boolean
```

T means “some type”.

The same function can safely work with different data types.

TypeScript can usually infer T automatically.

Avoid replacing generics with any when type safety matters.

25. Generics with Interfaces

A common pattern for API responses

```
interface ApiResponse<T> {
    success: boolean;
    data: T;
}

interface Student {
    id: number;
    name: string;
}

const response: ApiResponse<Student> = {
    success: true,
    data: { id: 1, name: "Hamim" }
};
```

Same wrapper, different data

```
ApiResponse<Student>

ApiResponse<Student[]>

ApiResponse<string>

ApiResponse<number>
```

Generics are especially useful when many API responses share the same structure but contain different data.

26. Putting Everything Together

A realistic TypeScript example

```
interface Student {
    id: number;
    name: string;
    cgpa?: number;
}

async function getStudents(): Promise<Student[]> {
    return [
        { id: 1, name: "Hamim", cgpa: 3.98 },
        { id: 2, name: "Rahim" }
    ];
}
```

Concepts used

✓ interface
✓ optional property (?)
✓ object literals
✓ Student[] array
✓ async function
✓ Promise<Student[]>

This style of typed asynchronous code is common in modern backend applications.

27. Common TypeScript Mistakes

Things students often get wrong

1. Wrong primitive type

```
let age: number = "25"; // ✕
```

2. Missing required property

```
interface User { id: number; name: string; }
const u: User = { id: 1 }; // ✕
```

3. Using any everywhere

```
let data: any = "hello";
data = 100;
data = true;
```

4. Confusing | with &

A | B means A OR B.

A & B means a value satisfying both types.

5. Forgetting that async functions return Promise<T>

28. TypeScript in NestJS

Why these concepts matter before learning NestJS

NestJS is built around TypeScript classes, decorators, dependency injection and typed application structures.

```
@Controller("students")
export class StudentController {
    constructor(
        private studentService: StudentService
    ) {}
}

@Injectable()
export class StudentService {
    async findAll(): Promise<Student[]> {
        // ...
    }
}
```

Before NestJS, students should be comfortable with:

- Types
- Interfaces / type aliases
- Functions
- Classes
- Constructors
- Access modifiers
- Inheritance
- async/await
- Promises
- Generics

29. Quick Reference Cheat Sheet

Keep this slide for revision

```
// Variable
let age: number = 25;

// Array
let names: string[] = ["A", "B"];

// Interface
interface Student { id: number; name: string; }

// Function
function add(a: number, b: number): number { return a + b; }

// Union
type ID = number | string;

// Optional
interface User { id: number; phone?: string; }
```

```
// Class
class Student {
  constructor(public name: string) {}
}

// Inheritance
class Graduate extends Student {}

// Async
async function getName():
Promise<string> {
  return "Hamim";
}

// Generic
function getData<T>(data: T): T {
  return data;
}
```

30. Exercises

Practice is more important than memorizing syntax

Exercise 1 — Student Interface

Create Student with id, name, email, cgpa and isActive. Create 3 students.

Exercise 2 — Functions

Write displayStudent(student: Student): void and print name + CGPA.

Exercise 3 — Generics

Write getLast<T>(items: T[]): T and test it with strings and numbers.

Exercise 4 — Async

Write getStudents(): Promise<Student[]> and use await to display the result.

Extra: combine interface + array + generic + async/await in one mini program.

31. Final Summary

The TypeScript mental model

1. TYPES

Describe what kind of data we have.

2. STRUCTURES

Interfaces, types, arrays and objects describe data.

3. BEHAVIOR

Functions and classes define what the program does.

```
Types
  ↓
Interfaces / Classes
  ↓
Functions + async/await
  ↓
Reusable type-safe application
  ↓
NestJS
```

Next lesson: NestJS fundamentals — project structure, modules, controllers, services and dependency injection.